

packages/graph-view/src/utils/layerStats.ts

Kind: added · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T20:01:22.578Z

Overview

`packages/graph-view/src/utils/layerStats.ts` is a new file that introduces a lightweight helper for summarizing a layer's complexity. It replaces an older, expensive node-filtering routine that was $O(N \times K)$ per layer.

Key changes

- **Imports** – `GraphNode` and `Layer` are imported from `@novadiff/graph-core/types` (line 1).
 - **Exports** –
 - `Complexity` type (`"simple" | "moderate" | "complex"`) – line 3.
 - `LayerStats` interface with `resolvedCount: number` and `aggregateComplexity: Complexity` – lines 5-10.
 - `computeLayerStats(layer: Layer, nodesById: Map<string, GraphNode>): LayerStats` – lines 20-39.
 - **Implementation** –
 - Iterates once over `layer.nodeIds`, looks up each node in `nodesById`.
 - Counts resolved nodes and tallies their `complexity` values.
 - Determines `aggregateComplexity` using a 30% threshold:``ts const aggregateComplexity: Complexity = counts.complex > resolved * 0.3 ? "complex" : counts.moderate > resolved * 0.3 ? "moderate" : "simple";
- `` - Returns `{ resolvedCount: resolved, aggregateComplexity }`.

Impact

- **Performance** – $O(\text{layer.nodeIds.length})$ versus the prior $O(N \times K)$ approach that caused freezes on large graphs.
- **Maintainability** – Centralizes complexity logic; future threshold changes can be made in one place.
- **Compatibility** – No API changes to existing modules; the new file is an addition only.

Risks & follow-ups

- Verify that all callers now use `computeLayerStats` instead of the old filter logic.
- Ensure every `GraphNode` in `nodesById` has a `complexity` property to avoid runtime errors.
- Confirm that the 30% threshold matches UI expectations; add tests for edge cases with low `resolvedCount`.
- Run lint, unit tests, and the production build to catch any type-safety or import issues introduced by the new file.